

TagGuard: Peripheral-to-Pipeline Hardware DIFT for Bare-Metal RISC-V

Ali Ait HASSOU

Univ. Montpellier

UMR CNRS 5506, LIRMM

Montpellier, France

ali.ait-hassou@umontpellier.fr

William PENSEC

Univ. Montpellier

UMR CNRS 5506, LIRMM

Montpellier, France

william.pensec@umontpellier.fr

Florent BRUGUIER

Univ. Montpellier

UMR CNRS 5506, LIRMM

Montpellier, France

florent.bruguiier@umontpellier.fr

Pascal BENOIT

Univ. Montpellier

UMR CNRS 5506, LIRMM

Montpellier, France

pascal.benoit1@umontpellier.fr

Abstract—Bare-metal IoT firmware is often exposed to untrusted bytes arriving from peripherals, which an attacker may control, and it runs without an operating system to label those bytes before they influence memory or control flow. Hardware Dynamic Information Flow Tracking (DIFT) is a natural defense, but prior systems either rely on software to create the first tag or leave part of the path from peripheral to core untagged. This weakens DIFT exactly where bare-metal devices are most exposed. We present TagGuard, an end-to-end DIFT integration in a CV32E40P-based SoC. TagGuard marks untrusted peripheral reads in hardware, carries tags through the AXI4-Lite interconnect and tags data memory, and then checks them in parallel with the data within the CV32E40P pipeline without stalls. A trusted startup security policy enables unmodified application firmware to run under a single configuration: trusted runtime routines may handle tagged input legitimately, while unexpected tagged control flow raises a security exception. On Xilinx Artix-7 XC7A200T, TagGuard adds 2.52% LUTs and 2.32% FFs relative to the base ADAM SoC configuration. It detects 16 bare-metal exploits across 15 distinct CWEs. Across all attacks and four legitimate workloads, we observe zero false negatives and zero false positives.

Index Terms—DIFT, RISC-V, SoC Security, Memory Tagging, Hardware Auto-Tagging, FPGA, CV32E40P

I. INTRODUCTION

Bare-metal IoT firmware often runs on small 32-bit microcontrollers without an operating system (OS) or Memory Management Unit (MMU). These devices now ship in tens of billions of units [1], and they may receive attacker-controlled bytes through peripheral interfaces such as UART, SPI, I2C, or GPIO. A single buffer overflow, format-string attack, integer overflow, or use-after-free can then turn external input into corrupted data or hijacked control flow. Software mitigations such as stack canaries, Address Space Layout Randomization (ASLR), and non-executable stacks help against specific exploit techniques, but they do not provide general data-flow protection.

Hardware Dynamic Information Flow Tracking (DIFT) addresses these attacks by attaching a tag to every byte at its entry point, propagating it through the pipeline as the data is computed or stored, and checking the tag before unsafe use. To protect legacy firmware, the DIFT tag must be created at the hardware boundary and must propagate through the full path: the memory system, the interconnect, and the core pipeline.

Prior DIFT systems leave part of this path outside the hardware. Core-level designs [2–4] focus on propagating and checking inside the processor, but assume that an OS or trusted firmware has already marked the input. System-level designs [5, 6] reach the interconnect but leave the peripheral boundary to software. Enclave mechanisms such as TIMBER-V [7] reuse tags for isolation rather than information flow tracking. Commercial memory-tagging extensions such as ARM MTE [8] attach metadata to pointers but do not propagate flow semantics across the SoC. These are reasonable design points for their targets, but they do not address the case of low-cost IoT systems where untrusted peripheral input is the primary attack source.

This paper closes that gap with TagGuard, an end-to-end DIFT extension demonstrated on an open-source RISC-V SoC, ADAM [9]. Tags are created at the peripheral boundary, transported through the AXI4-Lite interconnect and data memory, and checked in an extended CV32E40P pipeline [10]. A small hardware guard also suppresses tagged writes to protected Memory-Mapped Input/Output (MMIO) registers. Protected application firmware requires no per-byte application DIFT instrumentation; a trusted startup bundle configures the policy and provides trusted-range metadata before control is transferred to the firmware.

Contributions:

- 1) An I/O Security Unit for hardware-level peripheral auto-tagging and MMIO write protection, combined with an end-to-end tag transport using AXI4-Lite sideband user signals and per-byte memory storage.
- 2) An in-core DIFT extension for CV32E40P with a parallel tag register file, tag propagation, and tag checking running in parallel at zero pipeline stall cost.
- 3) A trusted startup bundle that protects unmodified application firmware without requiring per-byte tag instrumentation in the application code, while transparently handling legitimate trusted-runtime branches on tagged data.
- 4) Validation on Xilinx Artix-7 showing low area overhead and detection of 16 exploits across 15 distinct Common Weakness Enumeration (CWE) [11], without false positives on the evaluated legitimate workloads.
- 5) An open-source release of the complete TagGuard system

enabling reproducibility and community reuse [12].

The rest of this paper review related works (Sec. II), outlines the threat model (Sec. III), describes TagGuard’s architecture (Sec. IV), provides experimental evaluation (Sec. V), tracks limitations (Sec. VI), and concludes (Sec. VII).

II. RELATED WORKS

Hardware DIFT has been studied at four levels of abstraction: gate-level mechanisms, tagged memory architectures, processor-core extensions, and full SoC integration [13]. We survey each and highlight the gap at the peripheral boundary.

A. Gate-Level and Tagged-Memory Approaches

Tiwari et al. [14] proposed GLIFT, a gate-level technique that is sound by construction but reports up to 70 % area overhead, making it unattractive for a resource-constrained microcontroller. Tagged-memory work such as Efficient Tagged Memory [15] addresses tag caching and the memory hierarchy but does not address tag propagation inside the CPU or at the peripheral boundary. Song et al. [16] introduced HDFI on a modified Rocket core: a single tag bit enforces data-flow isolation at 2 % average overhead, but the design targets isolation rather than peripheral-input tag propagation and assumes privileged software to manage tags. TMDFI [17] brings related ideas to the lowRISC tagged-memory architecture for embedded targets but requires static analysis and reports 39 % runtime overhead.

B. Core-Level Approaches

Suh et al. [2] introduced one of the first architectural DIFT mechanisms: one tag bit per byte $\approx 1\%$ SPEC overhead, with peripheral tagging delegated to the OS: a workable assumption for managed systems but inapplicable for bare-metal IoT firmware. Raksha [3] generalized this with 4-bit tags and four concurrent software-defined policies; it is flexible but still assumes a privileged software layer and does not address tag initialization at the peripheral boundary. On RISC-V, Palmiero et al. [4] ported the approach to the RI5CY core with zero pipeline stalls and negligible area, including a per-byte memory tag bank, the closest in-core RISC-V predecessor to our pipeline. The SoC fabric and peripherals still remain outside the tagged path, and the security evaluation uses primarily the Wilander stack-attack suite [18].

C. SoC- and System-Level Approaches

Decoupled DIFT engines reduce core invasiveness by moving tag logic outside the main pipeline. Kannan et al. [19] present a dedicated coprocessor that caches 4-bit tags and synchronizes with the main core at system calls, reporting less than 1 % runtime overhead on SPECint2000. ARMHex [20] uses ARM CoreSight PTM, static analysis and selective instrumentation to recover the trace needed by an external DIFT engine on Zynq, reducing instrumentation overhead and protecting the monitor with TrustZone. These designs lower integration cost but still rely on traces, software assistance,

TABLE I: Comparison with prior hardware DIFT works.

Work	Platform	Core DIFT	Bus Tags	HW Auto-Tag	Periph. Guard	Unmod. App	CWE count
Suh [2]	Sim.	✓	x	x	x	x	2
Dalton [3]	FPGA	✓	x	x	x	x	6
Tiwari [14]	FPGA	✓	x	x	x	x	—
Porquet [5]	Sim.	x	✓	x	x	x	2
Song [16]	FPGA	✓	x	x	x	x	2
Wahab [20]	FPGA	✓	x	x	x	x	—
Palmiero [4]	FPGA	✓	x	x	x	x	4
Piccolboni [6]	FPGA	x	✓	x	x	x	2
Weiser [7]	Sim.	x	x	x	x	x	—
Chen [21, 22]	FPGA	✓	✓	x	✓	x	2
Liu [25]	FPGA	✓	x	x	x	x	4
This work	FPGA	✓	✓	✓	✓	✓	15

or system-call synchronization; they do not solve first-tag creation at an untrusted peripheral read.

Chen et al. [21, 22] implement an off-core DIFT coprocessor for Rocket-Chip that receives committed instructions via an extended RoCC interface; the SoC extension adds a dedicated TileLink tag bus with per-IP wrappers and MMIO tag registers at +10 % LUTs and +9.4 % FFs. However, the first tag must be created explicitly via custom coprocessor instructions, and policy setup requires source-level instrumentation at every I/O call site.

WHISK [5] and PAGURUS [6] transport tags across an uncore: WHISK with 2-bit tags on a NoC and an in-memory tag store (simulation only, peripheral boundary left to firmware) and PAGURUS by wrapping loosely-coupled accelerators in DIFT shells (protects accelerator paths but not the peripheral entry point). TIMBER-V [7] reuses tagged memory for enclave isolation rather than run-time tag tracking. Unlike pointer-safety mechanisms (ArmMTE, CHERI [23], HardBound [24]) that attach metadata to pointers or memory regions but do not propagate tags through the core pipeline, CCTAG [25] extends a Rocket core with configurable and combinable tag rules at 4.7–7.9 % SPEC overhead, but it targets Linux-class systems and does not create a tag at the peripheral boundary or carry it through the interconnect. HOPE [26] runs a programmable policy engine alongside the core and supports DIFT but reports 25–50 % area overhead, about an order of magnitude above our design. Pensec et al. [27] showed that the D-RI5CY tag logic is vulnerable to fault injection and proposed [28] a Hamming-code hardening at 10.6 % area overhead; this work is complementary and motivates the discussion in Section VI.

Table I summarizes architectural coverage. Area and runtime overheads are reported separately in Subsection V-A. The "Unmod. App" column means that the protected bare-metal application code needs no source changes or per-byte software tag instrumentation, although the final image may link trusted startup and policy metadata. As shown in Table I, TagGuard is unique in combining hardware auto-tagging, SoC-level transport, peripheral write-guarding, and support for unmodified legacy applications.

III. THREAT MODEL

We consider a bare-metal IoT device: a RISC-V firmware image runs on an in-order processor without an OS or MMU to

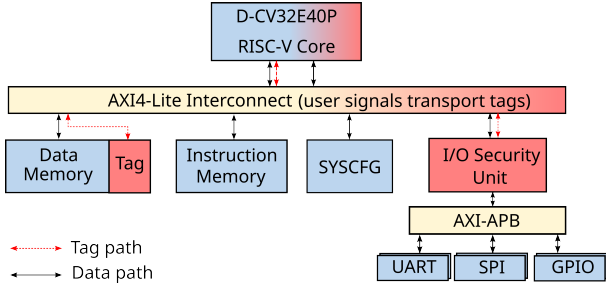


Fig. 1: ADAM SoC architecture with TagGuard integration.

mediate memory access or classify input. The device receives data through UART, SPI, and GPIO peripherals. We trust the processor, tag memory, tag-signal paths in the interconnect, peripheral tag source, MMIO guard, trusted startup code, DIFT handler, and trusted-range metadata.

Adversary. The adversary controls peripheral input bytes and may exploit buffer overflows, format-string attacks, integer errors, or pointer corruption in vulnerable firmware. The adversary cannot modify the firmware image, the trusted startup bundle, or the DIFT hardware; this reflects the standard embedded threat model where flash is write-protected at boot and hardware is trusted post-fabrication. Malicious firmware that deliberately rewrites DIFT policy registers after startup is outside the evaluated threat model.

Protection goal. TagGuard halts the application when tagged data reaches a checked branch, a tagged execute target, or a guarded MMIO write, and logs the event.

Scope. We evaluate CWE-derived bare-metal reproducers for control-flow hijacks, out-of-bounds accesses, format-string bugs, integer overflows, allocator misuse, uninitialized pointers, and guarded MMIO writes. Side channels, fault injection, and DMA-based attacks are out of scope, as the focus of this work is software-exploitable vulnerabilities.

IV. ARCHITECTURE

Figure 1 shows the ADAM SoC organization with TagGuard integration. The CV32E40P core with its DIFT mechanism (D-CV32E40P) is connected to instruction memory, data memory, configuration registers, and the peripheral subsystem through an AXI4-Lite interconnect. We keep this structure and add a tag path next to the normal datapath. The core propagates and checks tags (Subsection IV-A), data memory stores tags alongside program data (Subsection IV-B), the interconnect forwards tag bits on AXI user signals (Subsection IV-C), and the I/O Security Unit manages the peripheral boundary by tagging incoming reads and blocking tagged MMIO writes before the AXI-to-APB bridge (Subsection IV-D, Subsection IV-E).

A. In-Core DIFT Extension (D-CV32E40P)

The CV32E40P is a four-stage in-order RISC-V core. DIFT operates in three phases: a tag is set when untrusted data enter the SoC (Subsection IV-D), is propagated through computation and memory and is checked at security-sensitive uses; an exception is raised before any violating instruction commits.

Two custom CSRs define the DIFT policy. The Tag Propagation Register (TPR) specifies how tags propagate from source

operands to their destinations. The Tag Check Register (TCR) controls which tagged operands are security violations. By keeping these decisions separate, hardware can propagate tags broadly and check only selected uses.

Figure 2 shows the resulting parallel tag path, highlighted in red. Operand tags flow through pipeline registers alongside instruction operands, and each architectural register $x0-x31$ has a corresponding tag bit stored in a 32-entry tag register file parallel to the integer register file. A tag bit equal to 1 marks a byte as untrusted, while 0 denotes trusted data. The tag for register $x0$ is always zero. Forwarding uses the same EX→ID and WB→ID timing as the integer path, so tag propagation does not introduce additional pipeline stalls.

In ID, a *tag policy decoder* block decodes the instruction class and reads the TPR and TCR to produce the propagation mode and check-enable signals for that instruction; these control signals travel with the instruction operands into EX. In EX, the *Tag Propagation Logic* takes the source operand tags and the propagation mode from ID and computes the destination tag in parallel with the ALU; the *Tag Check Logic* takes the same operand tags and the check-enable signals and raises a synchronous exception before the WB stage if a violation is detected. For branches and jumps, a *PC Tag MUX* at the IF/ID boundary propagates the tag of the target address into the pipeline, where it feeds the *Execute Check* in EX. The LSU carries tags on a sideband: store tags leave with the write data, and load-response tags are OR-reduced across the returned bytes before writeback to the parallel tag register file.

1) *Tag Propagation Register (TPR)*: The TPR groups RV32 instruction classes and assigns each class a two-bit propagation mode (Table II). First mode, OLD (00), leaves the destination tag unchanged. Second mode, AND (01), sets the destination tag only when *both* source tags are set. Third mode, OR (10), sets the destination tag when *either* source tag is set; for example, an ADD whose first operand is tagged produces a tagged result even if the second operand is clean, conservatively tracking that malicious data influenced the result. Finally, CLEAR mode (11) always zeros the destination tag. The evaluated profile, $TPR=0x0000A8A2$, uses OR for integer, jump, shift, logical, and load/store instructions (propagating any source tag to the destination register), and OLD for branches and comparisons. For jump instructions (JALR), the destination register rd stores the return address, under the OR rule, and indirect jump propagates the tag of its source register to rd . Conditional branches write no register, so their propagation mode is immaterial; instead, control flow safety on a tagged branch is enforced by the *Branch Check* in the TCR. Among the three L/S Enable bits, only the value-operand bit (bit 15) is set; the load-base (bit 16) and store-destination (bit 17) address operands do not participate in propagation, restricting the spread of the tag to data values rather than addresses.

2) *Tag Checking Register (TCR)*: The TCR specifies which tagged operand uses are security violations. If an enabled check sees a tagged operand, the core raises a synchronous DIFT exception before the instruction is executed. As shown in

TABLE II: TPR bit fields. Reset value: 0x0000A8A2.

Bit index	L/S Enable 17 16 15	L/S Mode 13 12	Logical Mode 11 10	Comparison Mode 9 8	Shift Mode 7 6	Jump Mode 5 4	Branch Mode 3 2	Arith Mode 1 0
Policy	0 0 1	1 0	1 0	0 0	1 0	1 0	0 0	1 0

Modes: 00=OLD (keep t_d); 01=AND ($t_{s1} \wedge t_{s2}$); 10=OR ($t_{s1} \vee t_{s2}$); 11=CLR.

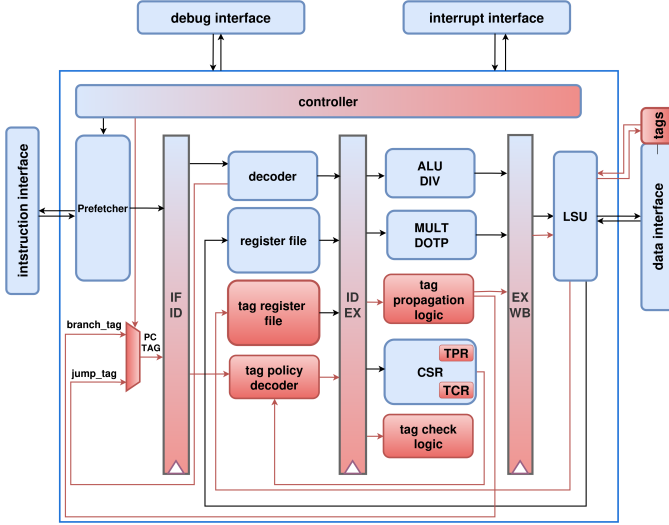


Fig. 2: D-CV32E40P architecture overview

Table III, the evaluated profile $TCR=0x00200018$ sets only two fields: Branch Check (bits 3–4=11) and Execute Check (bit 21=1); all other checks remain zero. The motivation for this pair: when an overflow corrupts a local variable, that variable inherits the attacker’s tag through load/store or arithmetic propagation; the first conditional branch that tests it supplies a tagged operand, triggering an exception before the branch outcome can be exploited. For control-flow hijacks, the corrupted return address or function pointer reaches a `jalr` target register, and the tagged-PC check fires before the first hijacked instruction can run. Software may select stricter profiles by enabling additional fields in Table III when the firmware policy requires it. For example, a pointer safety profile that additionally sets the Load/Store address checks (bits 17–20) traps the first use of tagged value as a load or store address, flagging a tainted pointer earlier rather than at a later branch; enabling the Comparison and Arithmetic checks tightens the policy toward a fully data integrity tracking, at a cost of a more frequent handler calls on legitimately tagged data.

B. Per-Byte Memory Tagging

TagGuard stores tags at byte granularity to match native RISC-V load/store resolutions; a single `LB` can load one byte of a word that the attacker has partially overwritten, so one tag bit per byte is the minimum resolution needed to track individual tagged bytes. Each 32-bit word therefore carries four tag bits. These four bits are transported through the SoC on the AXI `user` sideband signals (Subsection IV-C).

Figure 3 shows the physical layout: each address maps to a 36-bit memory word comprising 32 data bits and 4 tag bits. Data and tags are accessed together in a single memory

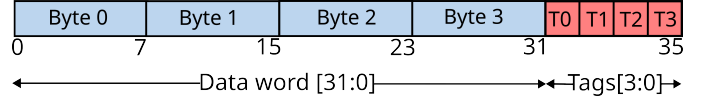


Fig. 3: Data memory word organization.

request, leaving latency unchanged. All tag bits are zero at power-on, so loads from uninitialized memory return a clean tag by default.

During a store, the byte-enable mask updates the selected data bytes and matching tag bits. On a load, the four returned tag bits are OR-reduced before writeback to the tag register file.

C. SoC-Level Tag Transport

Tags must cross the SoC wherever data crosses the SoC. Rather than widening the AXI data bus, TagGuard uses the optional AXI4-Lite `user` sideband to carry the four byte-lane tags introduced in Subsection IV-B. On the write path, `aw_user` carries the store-address tag, and `w_user` carries the four data-byte tags. On the read path, `r_user` returns the four-byte tags from data memory or the I/O Security Unit. The write-response channel (`b_user`) is forwarded transparently but carries no tag semantics. The main payload remains a standard 32-bit AXI transfer.

The interconnect, decoder, and arbiter forward all `user` bits transparently alongside every transaction. The I/O Security Unit intercepts `w_user` and `r_user` before the AXI-to-APB bridge; tag bits are consumed there, and APB, which carries no user signals, never sees them. Any slave that does not implement tag logic simply ignores the `user` bits it receives. Tag semantics are interpreted only at three endpoints: the D-CV32E40P core, the data memory, and the I/O Security Unit.

D. Hardware Auto-Tagging Unit

The first tag must appear the moment untrusted input enters the SoC. Software-based tagging requires the firmware to issue a custom tag-set instruction immediately after each peripheral read. This in turn demands a modified compiler toolchain and source-level annotations at every I/O call site. Even with that toolchain investment, a window exists between the load and the tag instruction during which the byte propagates as clean data. TagGuard eliminates both problems by tagging peripheral reads in hardware before the read response reaches the core.

The I/O Security Unit sits between the AXI4-Lite interconnect and the peripheral bus. On reads, it is controlled by the *DIFT Tag Source Register* (DTSR). Each DTSR bit designates one peripheral slot, a contiguous MMIO address range mapping one peripheral as a tag source. In the evaluated SoC, DTSR is five bits wide, covering one GPIO, one SPI, two timers, and one UART.

TABLE III: TCR bit fields. Reset value: 0x00200018.

Bit index	Execute Check 21	Load/Store Check 20 19 18 17	Logical Check 16 15 14	Comparison Check 13 12 11	Shift Check 10 9 8	Jump Check 7 6 5	Branch Check 4 3	Arith Check 2 1 0
Policy	1	0 0 0 0	0 0 0	0 0 0	0 0 0	0 0 0	1 1	0 0 0

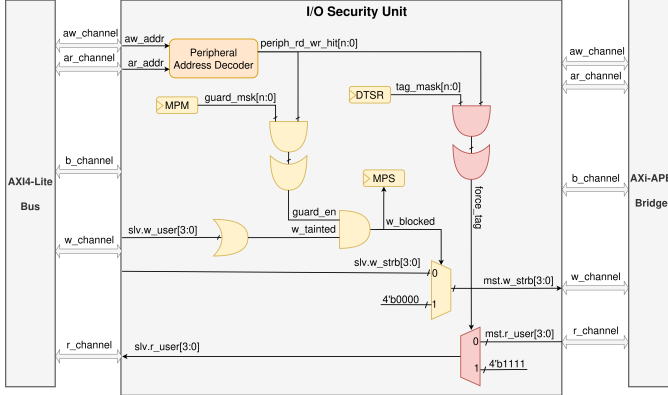


Fig. 4: I/O Security Unit architecture. Red: read path; Yellow: write path.

As Figure 4 shows, a Peripheral Address Decoder maps the incoming AXI read address to a slot index, which is then matched against the DTSR. Tagging applies only to a slot's *data* register. Any matching data-register read causes the unit to force the returned `r_user` to 4'b1111 on the same beat as the read data:

$$\text{slv.r_user} = \begin{cases} 4'b1111 & \text{if DTSR[slot(ARADDR)]} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

The tag arrives at the core in the same AXI read beat as its data, so no cycle window exists in which a byte is visible without its tag. Once trusted startup code sets DTSR, application firmware receives tainted input automatically.

E. MMIO Write Guard

Auto-tagging protects the path into the SoC. The write guard protects the path out to peripherals. This matters because an attacker who has already corrupted a value can try to turn that corruption into a visible effect by writing an MMIO register, such as an SPI control register, or a peripheral configuration.

The same I/O Security Unit that handles auto-tagging (Subsection IV-D) covers the outgoing direction. *Memory Protection Mask* (MPM) enables write guarding per peripheral slot, and the *Memory Protection Status* (MPS) records which slot triggered a block. The write address passes through the same Peripheral Address Decoder to identify the target slot; a write transaction is considered tagged when any byte-lane bit of `w_user` is asserted. When the MPM bit for that slot is set, the unit zeros the outgoing write strobes: the peripheral register value is unchanged, and the silently suppressed write has no effect on the hardware state. The AXI-to-APB bridge still completes a structurally valid transfer, so the CPU does not stall. The blocked slot index is recorded in MPS. A silent suppression is preferred over a bus exception to maintain strict AXI4-Lite protocol compliance and avoid pipeline stalls;

TABLE IV: Trusted runtime range categories.

Category	Purpose	Ranges
HAL I/O	UART and syscall wrapper loops	5
String / memory	Copy, scan, compare, and fill loops	12
Formatted I/O / parse	Formatting and numeric parsing	13
Allocator	Heap-metadata control flow	6
libgcc arithmetic	Compiler helper branches	5

instead hardware captures the violation in the MPS register, relying on software polling to handle recovery. This guard is exercised by Attack #16 (CWE-284, Subsection V-B).

Together, auto-tagging and the MMIO guard close the full datapath from peripheral input to security decision in both directions.

F. Trusted Startup Policy

The application firmware runs unmodified: no source changes or per-byte tag instrumentation are required. As Figure 5 shows, three DIFT sources (`dift_boot.c`, `dift_handler.S`, `dift_trusted.s`) are compiled alongside the unmodified application and linked with `libc.a/libgcc.a` by LLVM lld to produce the protected ELF.

Figure 6 shows the two operational phases. At boot, DIFTBoot executes as a constructor: it installs the exception handler, writes TPR/TCR and peripheral slot registers (DTSR/MPM), and registers the trusted-runtime table. All DIFT mechanisms are active before the `main` function is entered.

At runtime, DIFT violations and illegal instructions share the same exception vector (`mcause=2`). Deploying Branch Check on real firmware revealed false positives: standard library routines such as `strcpy`, `memcpy`, and `printf` legitimately branch on tagged data originating from peripherals. The handler resolves this by checking the faulting PC against 41 trusted ranges across five library categories (Table IV); a match causes the handler to resolve the branch and resume; otherwise, it records the violation and halts. Our evaluation assumes a baseline threat model where standard library code is trusted, classifying its data dependent loops as legitimate execution. However, bypassing checks within these boundaries leaves a security blind spot; an attacker can exploit input vulnerabilities that execute entirely inside these trusted routines without triggering a DIFT exception.

Algorithm 1 summarizes the full handler path. Registers are pushed onto a trap frame first to preserve operand values needed for branch resolution at step 11. TCR is zeroed immediately to disable DIFT checks inside the handler itself. The `find_branch` probe checks `mepc`, `mepc-2`, and `mepc-4` because the DIFT check fires in EX while ID has already advanced the PC. Branch resolution sets the next PC to the branch target if taken or to `br_pc + insn_len` if not taken.

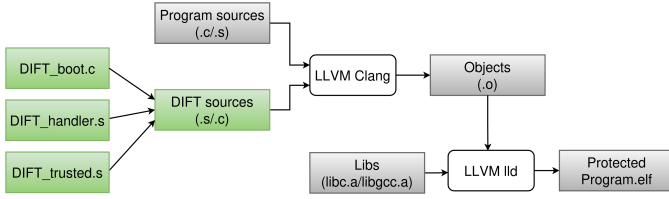


Fig. 5: TagGuard-aware compilation toolchain. Green: DIFT source files; grey: standard files and artifacts; white rounded boxes: LLVM tools.

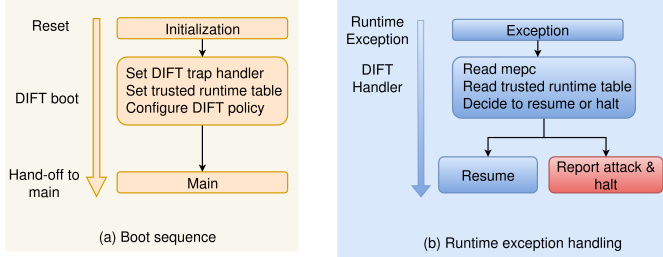


Fig. 6: TagGuard software integration.

V. EVALUATION

We evaluate TagGuard on a Digilent Nexys Video board (Xilinx Artix-7 XC7A200T), synthesized with Vivado 2025.2 at a 50 MHz system clock. Applications were compiled with Clang at `-O3` for `rv32imc_zicr`; disassembly was inspected to confirm that stack frame layout survives optimization. Hardware cost figures come from post-implementation Vivado reports. Attack detection and false-positive behavior are validated in simulation, where full pipeline visibility allows cycle-accurate inspection of tag propagation and exception signaling; representative cases are then confirmed on the physical board. We evaluate four aspects: hardware cost, attack coverage, false-positive behavior, and performance impact.

A. Hardware Overhead

Table V reports that TagGuard adds 617 LUTs (+2.52%), 311 flip-flops (+2.32%), and 49 BRAMs (+19.1%) at the SoC level. The BRAM increase is the physical manifestation of per-byte tag storage: each 32-bit word carries 4 tag bits (12.5% logical overhead), an artifact discussed in Section VI. Both builds close timing at 50 MHz with identical worst-case setup slack, yielding a zero-frequency penalty. The total power of the board increases by only 0.6% ($0.341\text{ W} \rightarrow 0.343\text{ W}$), driven entirely by static leakage in the additional BRAMs.

Table VI contextualizes these figures with previous work. The closest comparison is Palmiero [4] ($\approx 0.82\%$ LUTs at core level). TagGuard’s higher SoC-level figure reflects the added cost of tagging the peripheral boundary, whose security value is demonstrated in Subsection V-B. The 0% pipeline overhead reflects the hardware-path cost: tag propagation and checking run in parallel with the integer datapath at zero stall cost; trap and resolve handler costs are dependent on workload and reported separately in Subsection V-C.

Algorithm 1 DIFT exception handler.

```

1: push all registers onto trap frame on stack
2:  $\text{TCR} \leftarrow 0$ 
3:  $\text{pc} \leftarrow \text{mepc}$ 
4:  $(\text{br}, \text{br\_pc}) \leftarrow \text{find\_branch}(\{\text{pc}, \text{pc}-2, \text{pc}-4\})$ 
5: if  $\text{br} = \text{null}$  then
6:   halt
7: end if
8: if  $\text{br\_pc} \notin \text{trusted\_ranges}$  then
9:   record violation PC and halt
10: end if
11:  $\text{next\_pc} \leftarrow \text{resolve\_branch}(\text{br}, \text{trap frame})$ 
12:  $\text{mepc} \leftarrow \text{next\_pc}$ 
13: restore TCR and all registers from trap frame; mret

```

TABLE V: TagGuard resource utilisation.

Design	LUTs	Regs	BRAMs	ΔLUTs	ΔRegs	ΔBRAMs	ΔF_{max}
CV32E40P (base)	10 037	3 858	—	+1.76%	+3.73%	—	—
CV32E40P (DIFT)	10 214	4 002	—				
ADAM SoC (base)	24 437	13 388	257	+2.52%	+2.32%	+19.1%	0%
ADAM SoC (DIFT)	25 054	13 699	306				

B. Security Evaluation

We first walk through one concrete stack return-address attack, then generalize to the complete set of CWE-derived reproducers. In the protected runs, attacker-controlled input enters through UART and is tagged by hardware auto-tagging.

1) *Illustrative Attack Walk-through (Attack #2, CWE-121)*: Figure 7 contrasts the normal and attacked states, with the hardware detection path shown below.

Normal execution. The stack frame is intact: local variables occupy their allocated space, and the saved return address (RA) holds a legitimate pointer. The parallel tag register file mirrors the integer register file ($\times 0-\times 31$); all tag bits are zero (green), indicating that no register holds attacker-influenced data.

After overflow. During boot, trusted startup has enabled the UART source slot, so every byte received over the AXI read channel is automatically tagged. The attacker sends an oversized payload through UART. `strcpy()` copies it into the local buffer without bounds checking; the excess bytes overwrite the saved RA with the address of attacker-controlled shellcode. Because the payload carries `tag=1` from the UART source, the load/store propagation rule causes every store to copy the source tag into the tag bank alongside the data byte. The saved RA slot in the stack frame is therefore stored in memory with `tag=1`. When the function loads RA into `x1` (the RISC-V return address register), the tag follows into the parallel tag file.

Detection. The `jblr x1` instruction uses `x1` as its jump target. Since `Execute Check` is set in the TCR, the DIFT hardware evaluates the tag of `x1` in the EX stage. Since `x1` is tagged, a violation is raised before the shellcode is executed.

2) *Common Policy and CWE Coverage*: Every attack reproducer uses the same startup configuration:

TABLE VI: Headline overheads of representative hardware DIFT works.

Work	Platform	Hardware / storage	Runtime
Suh [2]	Sim	+1.44% mem	+1.1% SPEC
Dalton [3]	FPGA	+7.17% LUTs; 12.5% tags	—
Porquet [5]	Sim	—	3.5–7.25%
Palmiero [4]	FPGA	≈0.82% LUTs; 12.5% tags	—
Song [16]	FPGA	—	+2% avg
Chen [22]	FPGA	+10% LUTs; +9.4% FFs; 12.5% tags	+3% avg
Liu [25]	FPGA	—	4.71–7.93% SPEC
This work	FPGA	+2.52% LUTs; +2.32% FFs; 12.5% tags	0%

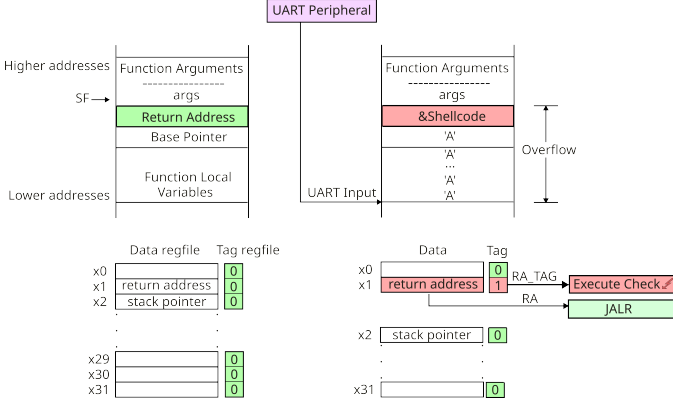


Fig. 7: Stack overflow illustration.

TPR=0x0000A8A2 and TCR=0x00200018. UART hardware auto-tagging creates the first tag without source-level application instrumentation, and the trusted-runtime table from Subsection IV-F covers trusted library branches. Each reproducer is first run without DIFT to confirm exploitability, then re-run with DIFT enabled.

All 16 CWE-derived reproducers are stopped (Table VII). The three control-flow hijacking attacks trigger *Execute Check*: tagged data lands in a `jalr` target register, and the pipeline raises an exception before the first hijacked instruction can be executed (the shellcode never runs). The twelve data-corruption and memory-error attacks trigger *Branch Check*: the corrupted or overflowed value eventually reaches a program branch that tests it, and the tag on that operand flags the violation before the branch outcome can be exploited. Attack #16 is stopped at the bus level: the MMIO guard intercepts the unauthorised peripheral write before it reaches the hardware register, and the write is silently dropped.

3) *False Positives (FP)*: Stopping attacks is only the beginning: a detector with benign alarms is unusable. We therefore ran four legitimate workloads that cover untagged compute, tagged arithmetic, and tagged-input DIFTBoot paths (Table VIII). We count a false positive as an unintended *attack halt*, not a legitimate (handled) branch resolution that transparently resumes execution. MatMul and FFT verify that DIFT does not corrupt untagged computation. The UART checksum workload confirms tagged bytes propagate correctly through arithmetic. The UART command parser exercises eight branch resolutions on tagged data, all transparently resumed. The firmware with libc/allocator workload drives auto-tagged input through a representative libc and malloc path, producing 107

TABLE VII: Attack demonstration suite.

Attack #	Exploit target	CWE	Detector
<i>Control-flow hijacking</i>			
1	BSS buffer overflow	120	EXECUTE
2	Stack RA overwrite (ROP)	121	EXECUTE
3	Func. ptr. overwrite	787	EXECUTE
<i>Data corruption and memory errors</i>			
4	OOB index write	119	BRANCH
5	Stack variable corrupt.	121	BRANCH
6	Heap overflow	122	BRANCH
7	OOB read	125	BRANCH
8	Format string %n	134	BRANCH
9	Int. overflow bypass	190	BRANCH
10	Off-by-one error	193	BRANCH
11	Double free	415	BRANCH
12	Use-after-free	416	BRANCH
13	Null deref.	476	BRANCH
14	strcpy BOF	676	BRANCH
15	Uninit. pointer use	824	BRANCH
<i>MMIO write guard</i>			
16	Periph. ctrl. reg. write	284	MMIO guard

branch resolutions without an attack halt.

TABLE VIII: Legitimate workloads used for false-positive evaluation.

Workload	Test goal	Outcome
MatMul / FFT	No side-effects on clean data	Correct; 0 FP
UART checksum	Tags propagate correctly	Match; 0 FP
UART command parser	Transparent resolution	8 resol.; 0 FP
Firmware with libc/allocator	Tags survive libe/malloc path	107 resol.; 0 FP

C. Performance Evaluation

The trusted startup code configures the DIFT handler, propagation and checking CSRs, peripheral tag-source bits, and MMIO-guard bits before `main`. In simulation we measure this one-time setup at 62 cycles, independent of application size or workload. Separate from this boot cost, tag propagation and checking after `main` run in parallel with the integer datapath and introduce no measurable steady-state overhead.

For workloads that process tagged input, overhead is dominated by the handler cost. Each handler invocation requires on average 274–458 cycles (5.5–9.2 μ s at 50 MHz). Table IX shows measured cycle counts for the two workloads cited previously.

The 9.79% overhead is a worst-case figure: it arises in a microbenchmark that deliberately drives all 107 branch resolutions through tightly looped libc string routines. Real applications with more computation will see proportionally lower overhead.

VI. DISCUSSION

The evaluation results confirm that TagGuard achieves its two primary goals: end-to-end DIFT coverage from peripheral input to pipeline check at low hardware cost, and protection of unmodified application firmware under a single unified boot-time configuration. We now discuss the main trade-offs, limitations, and directions for future work.

The 12.5% logical tag-storage overhead is intrinsic to per-byte tag granularity. Since tagged bytes are statistically rare

TABLE IX: Trap and resolve handler overhead.

Workload	Handler cycles	Total cycles	Overhead
UART command parser	2 196	402 929	0.54%
Firmware with libc/allocator	49 097	501 440	9.79%

in practice, a demand-paged tag structure allocating tag pages only on first use would scale storage with actual tag spread rather than total address space size.

Application firmware requires no modification beyond a trusted startup bundle. The practical limit of this approach is trusted table completeness: any trusted routine absent from the table that branches on tainted data will trigger a false positive. Automating table construction via binary analysis or profiling is the natural next step toward broader applicability.

The security I/O unit intercepts peripheral data on the CPU bus path, so coverage is guaranteed only for CPU-driven transfers. DMA-capable SoCs represent the primary architectural gap: bytes moved directly from a peripheral FIFO into memory bypass the tagging unit entirely. Attaching tagging logic to the DMA read channel is the main direction for future work on production SoCs.

Finally, the DIFT control registers themselves are a fault-injection target. Pensec et al. [27, 28] demonstrated that a single bit-flip in TCR or TPR silently disables protection with no processor exception; their Hamming-code hardening approach applies directly to our CSRs. Quantifying the resilience cost of hardening both remains an open question.

VII. CONCLUSION

We presented TagGuard, an end-to-end hardware DIFT integration for the ADAM RISC-V SoC that closes the full peripheral-to-pipeline tag path without modifying application firmware. An I/O security unit tags untrusted peripheral reads and blocks tagged MMIO writes; tags ride the AXI4-Lite user signals through the interconnect, are stored per-byte in tag memory, and are propagated and checked in parallel with the integer datapath inside the CV32E40P core. A trusted startup bundle configures the full policy before `main` is entered.

On a Xilinx Artix-7 XC7A200T, TagGuard adds 2.52% LUTs and 2.32% FFs at SoC level, with a one-time 62-cycle boot cost and no pipeline stall overhead. It detects all sixteen CWE-derived reproducers across 15 distinct CWE classes with zero false negatives and zero false positives on the evaluated legitimate workloads.

Future work will focus on extending auto-tagging to DMA-driven transfers, automating trusted table construction, and hardening the policy registers against fault injection.

ACKNOWLEDGMENT

This work was supported by the French National Research Agency (ANR) through the government grant SCREAM (ANR-24-CE39-0199-02).

REFERENCES

- [1] IoT Analytics. *State of IoT 2024: Number of connected IoT devices growing to 18.8 billion globally*. <https://iot-analytics.com/number-connected-iot-devices/>. Projected to exceed 30 billion by 2030. 2024.
- [2] G. Suh et al. “Secure Program Execution Via Dynamic Information Flow Tracking”. In: (Dec. 2004). DOI: 10.1145/1037947.1024404.
- [3] Michael Dalton, Hari Kannan, and Christos Kozyrakis. “Raksha: A Flexible Information Flow Architecture for Software Security”. In: June 2007. DOI: 10.1145/1250662.1250722.
- [4] Christian Palmiero et al. “Design and Implementation of a Dynamic Information Flow Tracking Architecture to Secure a RISC-V Core for IoT Applications”. In: Sept. 2018. DOI: 10.1109/HPEC.2018.8547578.
- [5] Joel Porquet and Simha Sethumadhavan. “WHISK: An uncore architecture for Dynamic Information Flow Tracking in heterogeneous embedded SoCs”. In: Sept. 2013. DOI: 10.1109/CODES-ISSS.2013.6658991.
- [6] Luca Piccolboni, Giuseppe Di Guglielmo, and Luca Carloni. *PAGU-RUS: Low-Overhead Dynamic Information Flow Tracking on Loosely Coupled Accelerators*. Dec. 2019. DOI: 10.48550/arXiv.1912.11153.
- [7] Samuel Weiser et al. “TIMBER-V: Tag-Isolated Memory Bringing Fine-grained Enclaves to RISC-V”. In: Jan. 2019. DOI: 10.14722/ndss.2019.23068.
- [8] Arm Ltd. *Arm® Architecture Reference Manual Supplement—The Memory Tagging Extension to the ARMv8-A Architecture*. Specification DDI 0614. 2019.
- [9] Felipe Paiva Alencar et al. “ADAM: ADaptive Microcontroller Platform for Edge AI Systems”. In: 2026. DOI: 10.1145/3768617.3770731.
- [10] OpenHW Group. *CV32E40P RISC-V Core*. <https://github.com/openhwgroup/cv32e40p>. 2023.
- [11] MITRE Corporation. *Common Weakness Enumeration (CWE)*. <https://cwe.mitre.org>. 2024.
- [12] Ali Ait Hassou et al. *GitLab ADAM DIFT*. <https://github.com/aliaithassou34/tagguard>. Published with camera-ready paper. 2026.
- [13] Christopher Brant et al. “Challenges and Opportunities for Practical and Effective Dynamic Information Flow Tracking”. In: (Nov. 2021). DOI: 10.1145/3483790.
- [14] Mohit Tiwari et al. “Complete Information Flow Tracking from the Gates Up”. In: (Mar. 2009). DOI: 10.1145/1508284.1508258.
- [15] Alexandre Joannou et al. “Efficient Tagged Memory”. In: 2017. DOI: 10.1109/ICCD.2017.112.
- [16] Chengyu Song et al. “HDFI: Hardware-Assisted Data-flow Isolation”. In: May 2016. DOI: 10.1109/SP.2016.9.
- [17] Tong Liu et al. “TMDFI: Tagged Memory Assisted for Fine-Grained Data-Flow Integrity Towards Embedded Systems Against Software Exploitation”. In: Aug. 2018. DOI: 10.1109/TrustCom/BigDataSE.2018.00083.
- [18] John Wilander and Mariam Kamkar. “A Comparison of Publicly Available Tools for Dynamic Buffer Overflow Prevention.” In: Jan. 2003.
- [19] Hari Kannan, Michael Dalton, and Christos Kozyrakis. “Decoupling Dynamic Information Flow Tracking with a Dedicated Coprocessor”. In: Aug. 2009. DOI: 10.1109/DSN.2009.5270347.
- [20] Muhammad Wahab et al. “ARMHEx: A hardware extension for DIFT on ARM-based SoCs”. In: Sept. 2017. DOI: 10.23919/FPL.2017.8056767.
- [21] Kejun Chen et al. “Hardware and Software Co-Verification from Security Perspective”. In: Dec. 2019. DOI: 10.1109/MTV48867.2019.00018.
- [22] Kejun Chen, Lei Sun, and Qingxu Deng. “Hardware and software co-verification from security perspective in SoC platforms”. In: (2022). DOI: <https://doi.org/10.1016/j.sysarc.2021.102355>.
- [23] Robert Watson et al. “CHERI: A Hybrid Capability-System Architecture for Scalable Software Compartmentalization”. In: (May 2015). DOI: 10.1109/SP.2015.9.
- [24] Joe Devietti et al. “Hardbound: architectural support for spatial safety of the C programming language”. In: (Mar. 2008). DOI: 10.1145/1353535.1346295.
- [25] Zhanpeng Liu et al. “CCTAG: Configurable and Combinable Tagged Architecture”. In: Jan. 2025. DOI: 10.14722/ndss.2025.240862.
- [26] Gregory Sullivan et al. “The Dover inherently secure processor”. In: Apr. 2017. DOI: 10.1109/THS.2017.7943502.
- [27] William Pensec, Vianney Lapôte, and Guy Gogniat. “Another Break in the Wall: Harnessing Fault Injection Attacks to Penetrate Software Fortresses”. In: Nov. 2023. DOI: 10.1145/3628356.3630116.
- [28] William Pensec et al. “Defending the Citadel: Fault Injection Attacks Against Dynamic Information Flow Tracking and Related Countermeasures”. In: July 2024. DOI: 10.1109/ISVLSI61997.2024.00042.